Name:

Stephen Ugbana

Project Title

Hospital-Based Data Warehouse and OLAP Solution

About the Project

This project developed a strategic healthcare data warehouse and business intelligence solution to support hospital management in analysing admissions and treatment-related performance. An ADMISSION centred Dimensional Fact Model (DFM) was developed from functional dependencies using attribute-tree construction, pruning, and grafting. The resulting star-schema data warehouse was implemented in SQLite, with OLAP queries and a consolidated materialised view enabling efficient analysis of charges per operation, length of stay and admission cost by Ward, Year and Quarter.

Skills

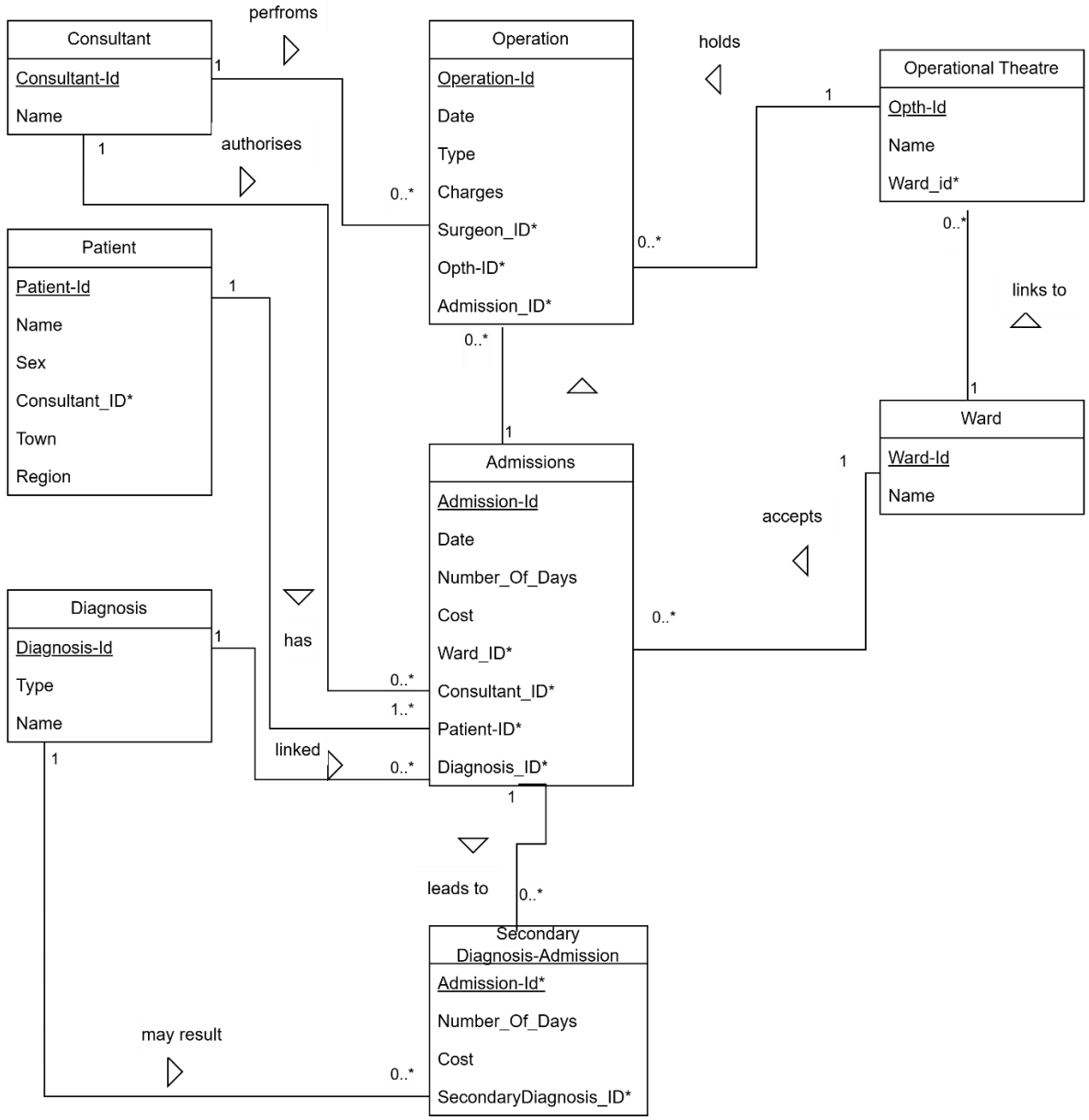
Data Warehousing · Business Intelligence · Dimensional Modelling · DFM Modelling · Star Schema Design · Functional Dependency Analysis · Attribute Trees · Pruning & Grafting · OLAP Analysis · SQL · SQLite · Python · Materialised Views · Query Optimisation · Data Aggregation · Database Design · Data Modelling · Analytical Reporting · Business Performance Analysis

Github

https://github.com/StephenCOUgbana/hospital-data-warehouse-and-bi-analytics

Problem Specification

The entity-relationship (ER) model of a simplified hospital domain, below, shows Primary keys underlined and foreign keys indicated by a star (*) next to the relevant attributes.



The Hospital authorities would like to build a strategic data mart to address frequent queries. These queries are:

Q1. For each Ward, Year and Quarter, report the charges per operation, length of stay and admissions cost;

Q2. For each Ward, Year, and Quarter, report the charges per operation, length of stay, and admission cost for Secondary Diagnosis.

Executive Summary

This project focused on the design and implementation of a strategic hospital data warehouse to support managerial analysis of hospital admissions. The solution addresses the primary business requirement to report, for each Ward, Year, and Quarter, the charges per operation, length of stay, and admission cost, both for overall and Secondary Diagnosis.

Methods

An ADMISSION fact was identified in the source entity-relationship model and used as the central business event. Functional dependencies were analysed to identify the relevant warehouse entities, followed by the construction, pruning and grafting of the Attribute Tree. The resulting Dimensional Fact Model (DFM) identified the appropriate dimensions, facts and measures. The DFM was subsequently implemented as a SQLite star schema data warehouse with dimension and fact tables. Sample data was then inserted to validate the implementation. Based on the OLAP analysis, a consolidated materialised view table was designed and implemented at the common Ward–Year–Quarter aggregation level to efficiently answer the frequent queries.

Results

The implemented warehouse successfully answers the frequent managerial queries. The materialised view precomputes overall and Secondary Diagnosis aggregates, allowing charges per operation, length of stay and admission cost to be retrieved efficiently without repeatedly aggregating the admission-level fact table. SQL queries were executed against the materialised view, producing the required Ward–Year–Quarter results.

Conclusion

This project provides a structured and scalable analytical solution for hospital management. The DFM, SQLite implementation, and consolidated materialised views align the data warehouse solution with a hospital's typical business requirements while reducing the need for repeated aggregation to answer frequent managerial queries.

1. Integrated DFM Design

1.1 Introduction

The data warehouse design has been developed using a well-established Dimensional Fact Model (DFM) methodology. Thus, the design begins by identifying the relevant transaction, component, and classification entities, then constructs an attribute tree and derives the final DFM scheme.

This solution chooses **ADMISSION** as the primary root fact. This is the most appropriate choice because the required analysis concentrates on hospital admission episodes, enabling the most common queries, grouped by Ward, Year, and Quarter, to be answered. Furthermore, the other transaction entities, OPERATION and SECONDARY_DIAGNOSIS_ADMISSION, are linked to ADMISSION through one-to-many relationships.

1.2 Solution to Tasks

A) **The Transaction Entities and the corresponding functional dependencies:** These are the dynamic business processes that occur over time and are candidates for facts.

The source ER model contains the following transaction entities:

- **ADMISSIONS:** The core business event and the chosen root fact for the DFM.

Functional dependency: Admission_Id → Date, Number_Of_Days, Cost, Ward_Id, Consultant_Id, Patient_Id, Diagnosis_Id

- **OPERATION:** is a transaction entity because charges per operation can be derived from this entity.

Functional dependency: Operation_Id → Date, Type, Charges, Surgeon_Id, Opth_Id, Admission_Id

- **SECONDARY_DIAGNOSIS_ADMISSION:** is a transaction entity because it represents the relationship between an admission and any secondary diagnosis.

Functional dependency: Admission_Id → SecondaryDiagnosis_Id Number_Of_Days, Cost.

B) The Component Entities and the corresponding functional dependencies: These are the dimensions that explain the admission event. They are the who / what / when / where / why of the business event.

The source ER model contains the following component entities:

- **WARD**: Ward_Id → Ward_Name
- **CONSULTANT:** Consultant_Id → Consultant_Name

- **PATIENT:** Patient_Id → Patient_Name, Sex, Consultant_Id, Town, Region
- **DIAGNOSIS:** Diagnosis_Id → Diagnosis_Name, Diagnosis_Type
- **OPERATIONAL THEATRE:** Opth_Id → Opth_Name, Ward_Id
- **TIME** (derived component dimension): Date → Month, Quarter, Year

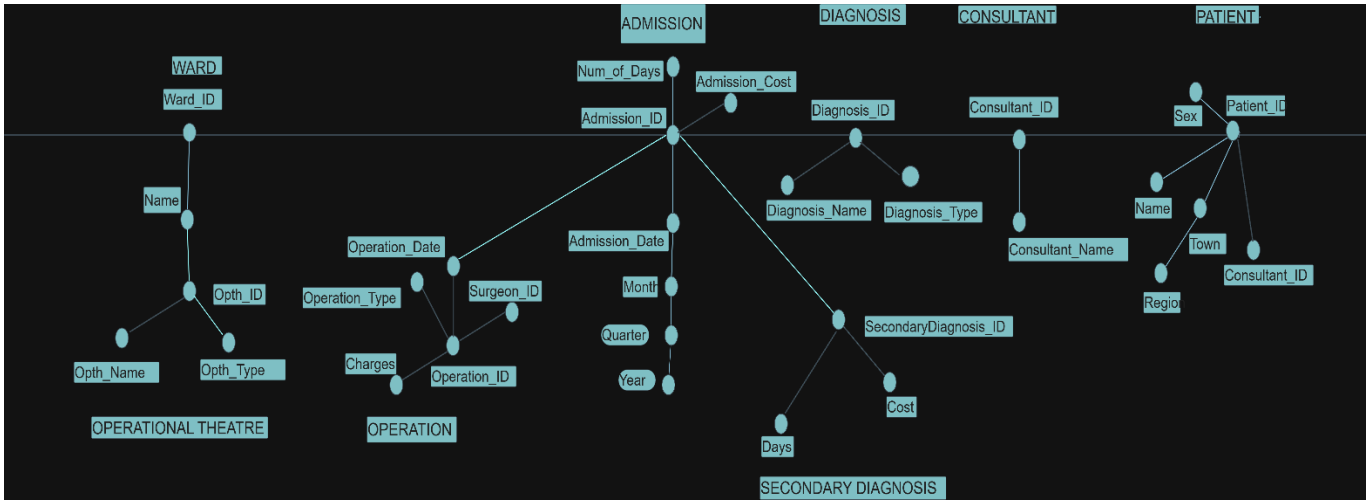
C) The Classification Entities (Hierarchies) and their functional dependencies:

- **Time hierarchy**: Date → Month → Quarter → Year
- **Diagnosis hierarchy**: Diagnosis_Name → Diagnosis_Type
- **Operation hierarchy**: Operation_ID → Opth_ID → Ward_Id
- **Patient geography hierarchy**: Town → Region

D) The Attribute Tree

- **Construction of the Attribute Tree from the ER Diagram**

The construction of the Attribute Tree was designed to include all attributes that are functionally dependent on ADMISSIONS through the various many-to-one relationships. The only exception is OPERATIONAL THEATRE, which, from the ERD, is not directly linked to ADMISSIONS. Therefore, it is represented in the Attribute Tree as linked to WARD, as shown in the ERD. Given below is the Attribute Tree.

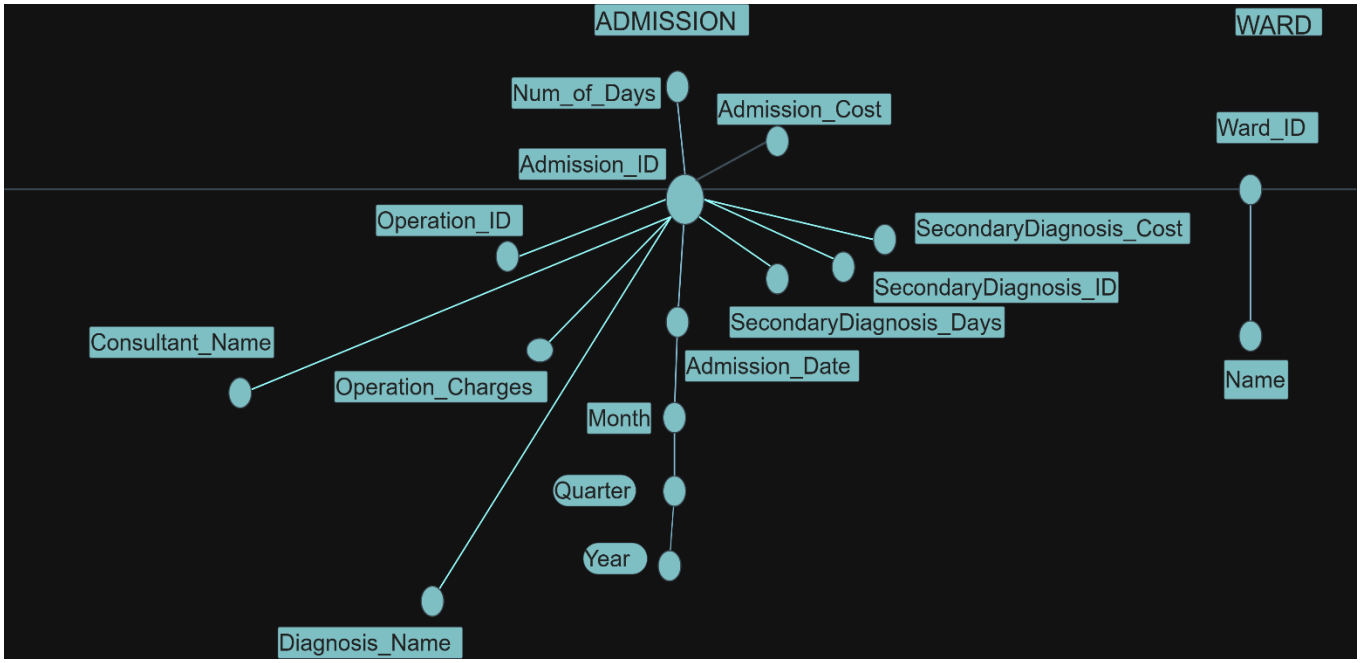


- **Pruning and Grafting of the Attribute Tree**

Pruning: The pruning has removed the descriptive branches that do not support the workload required by the hospital authorities. Specifically, the earlier branches for OPERATIONAL THEATRE and PATIENT have been taken out.

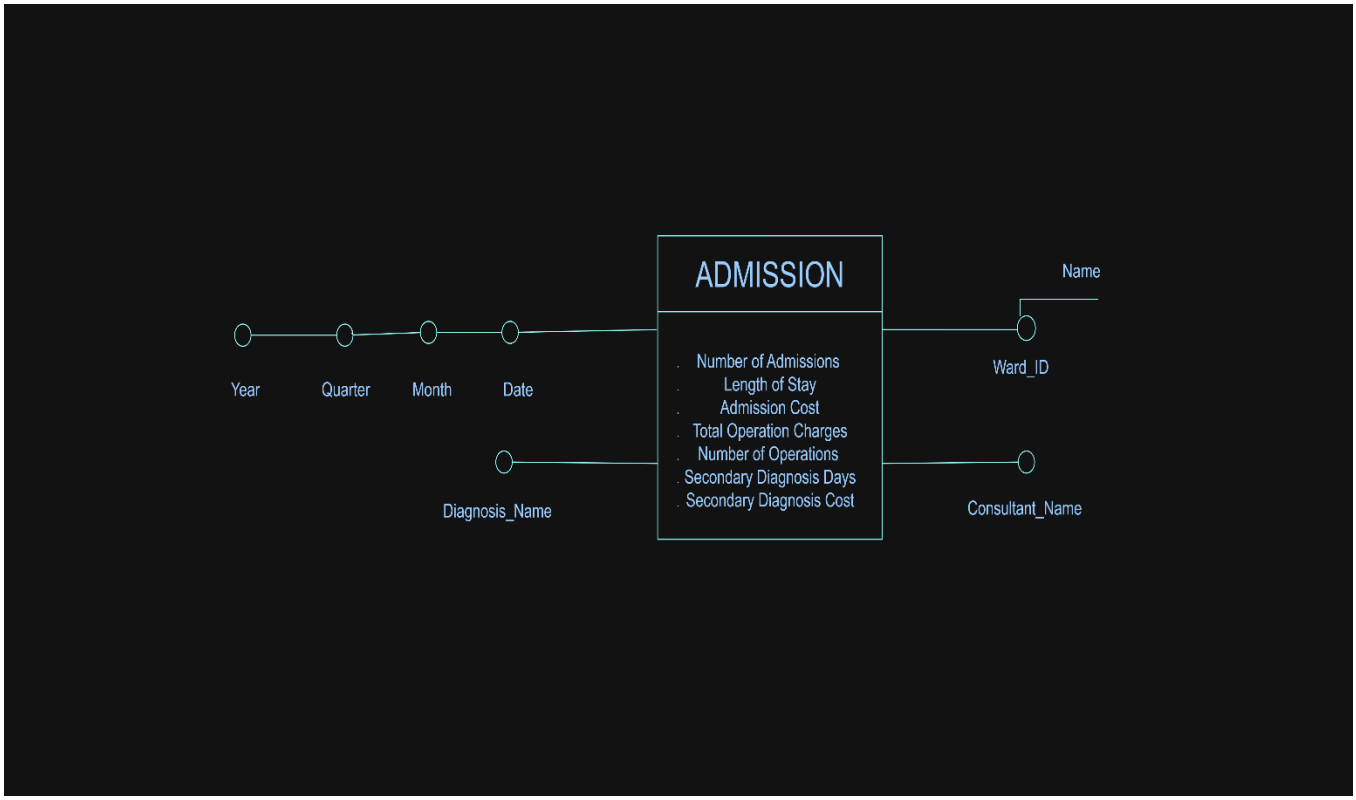
Grafting: The grafting has removed the descriptive attributes in the OPERATION and SECONDARY DIAGNOSIS branches, reducing them to the numerical attributes required by the queries. Additionally, the descriptive attributes in the CONSULTANT and DIAGNOSIS branches have been removed.

This approach for pruning and grafting is suitable because it reduces the attribute tree to the necessary measures that answer the queries for Ward, Year, and Quarter on charges per operation, length of stay, admission cost, and secondary diagnosis. Thus, given below is the pruned and grafted attribute tree:



E) The DFM schema derived from the Updated Attribute Tree

Find below the DFM Schema:



- **Dimensions:** The final dimensions and their hierarchy derived from the updated attribute tree are:

  - **Time:** Month → Quarter → Year

These are the correct dimensions because the problem specification requires both queries to be answered for each Ward, Year, and Quarter. Therefore, these are the true analytical viewpoints from which the fact must be aggregated.

Note*: In the final DFM, Admission_Date is the underlying date attribute from which the time hierarchy of Month–Quarter–Year is derived*.

- **Fact:** The fact remains **ADMISSION**.

This is the correct fact because the business event being analysed is the hospital admission episode. The required measures in both queries, which are length of stay, admission cost, and charges per operation, are all naturally centred on an admission.

- **Measures**: The DFM measures were obtained by defining fact attributes as counts of fact instances or as aggregations over numerical attributes that remain after the dimensions have been selected. Thus, applying this principle to the updated attribute tree, the DFM measures are:

  - Number of Admissions = COUNT(ADMISSION)
  - Length of Stay = SUM(Num_of_Days)
  - Admission Cost = SUM(Admission_Cost)
  - Total Operation Charges = SUM(Operation_Charges)
  - Number of Operations = COUNT(Operation_ID)
  - Secondary Diagnosis Days = SUM(SecondaryDiagnosis_Days)
  - Secondary Diagnosis Cost = SUM(SecondaryDiagnosis_Cost)

The key derived KPI required by both queries is:

  - Charges per Operation = SUM(Operation_Charges) / COUNT(Operation_ID)

This is the correct formulation because Q1 and Q2 explicitly ask for charges per operation, not simply total charges. In the final DFM, this should be treated as a derived measure, calculated from the stored additive measures rather than stored directly as a base fact attribute.

For Query 2, which asks for the same measures “of Secondary Diagnosis”, the model can answer the query by using the same Ward and Time dimensions while focusing on admissions associated with secondary diagnosis. The inclusion of SecondaryDiagnosis_Days and SecondaryDiagnosis_Cost strengthens the model by providing a richer analytical basis for the secondary-diagnosis component of the workload.

- **Overall Correctness of the Delivered DFM Model:**

Overall, the delivered DFM model is correct. Its main strengths are that it correctly identifies ADMISSION as the central fact, retains the Time dimension explicitly required by the workload, and includes the numerical attributes necessary to answer both Queries 1 and 2. The delivered DFM is therefore well aligned with the managerial queries in the problem specification.

2. Implemented DFM Schema

1.1 Introduction

Below is a SQLite implementation of the DFM schema. It is designed so the warehouse can answer Queries 1 and 2 by aggregating admissions by Ward, Year, Quarter, while storing the required admission, operation, and secondary-diagnosis measures in a single admission-grain fact table.

In this implementation:

- The fact is ADMISSION.
- The fact grain is one row per admission.

The dimensions implemented from the DFM are Time, Ward, Consultant, and Diagnosis

The measures are stored in the fact table:

- number of admissions
- length of stay
- admission cost
- total operation charges
- number of operations
- secondary diagnosis days
- secondary diagnosis cost

1.2 Implementation Code for the DFM Schema

```
# Colab-ready SQLite schema implementation for the DWBI data warehouse

import sqlite3
from pathlib import Path

# Create/connect to SQLite database file
db_path = "/content/hospital_dw.db"
conn = sqlite3.connect(db_path)
cur = conn.cursor()

# Enable foreign keys
cur.execute("PRAGMA foreign_keys = ON;")
```

```
# Drop existing objects if they already exist
drop_sql = """
DROP VIEW IF EXISTS vw_q1_ward_year_quarter;
DROP VIEW IF EXISTS vw_q2_ward_year_quarter_secondarydiagnosis;

DROP TABLE IF EXISTS fact_admission;
DROP TABLE IF EXISTS dim_diagnosis;
DROP TABLE IF EXISTS dim_consultant;
DROP TABLE IF EXISTS dim_ward;
DROP TABLE IF EXISTS dim_time;
"""
cur.executescript(drop_sql)

# Create schema
schema_sql = """
PRAGMA foreign_keys = ON;

-- ========================
-- DIMENSIONS
-- ========================

CREATE TABLE dim_time (
    time_key        INTEGER PRIMARY KEY,
    full_date       TEXT NOT NULL UNIQUE,      -- YYYY-MM-DD
    month_num       INTEGER NOT NULL CHECK (month_num BETWEEN 1 AND
12),
    quarter_num     INTEGER NOT NULL CHECK (quarter_num BETWEEN 1 AND
4),
    year_num        INTEGER NOT NULL
);

CREATE TABLE dim_ward (
    ward_key        INTEGER PRIMARY KEY,
    ward_id_bk      INTEGER NOT NULL UNIQUE,
    ward_name       TEXT NOT NULL
);

CREATE TABLE dim_consultant (
    consultant_key      INTEGER PRIMARY KEY,
    consultant_name     TEXT NOT NULL UNIQUE
);

CREATE TABLE dim_diagnosis (
    diagnosis_key       INTEGER PRIMARY KEY,
    diagnosis_name      TEXT NOT NULL UNIQUE
);
```

```
-- ========================
-- FACT TABLE
-- Grain: one row per admission
-- ========================

CREATE TABLE fact_admission (
    admission_key                   INTEGER PRIMARY KEY,
    admission_id_bk                 INTEGER NOT NULL UNIQUE,

    time_key                        INTEGER NOT NULL,
    ward_key                        INTEGER NOT NULL,
    consultant_key                  INTEGER,
    diagnosis_key                   INTEGER,

    -- Measures
    number_of_admissions            INTEGER NOT NULL DEFAULT 1 CHECK
(number_of_admissions >= 0),
    length_of_stay                  INTEGER NOT NULL CHECK
(length_of_stay >= 0),
    admission_cost                  REAL NOT NULL CHECK (admission_cost
>= 0),

    total_operation_charges         REAL NOT NULL DEFAULT 0 CHECK
(total_operation_charges >= 0),
    number_of_operations            INTEGER NOT NULL DEFAULT 0 CHECK
(number_of_operations >= 0),

    secondary_diagnosis_days        INTEGER NOT NULL DEFAULT 0 CHECK
(secondary_diagnosis_days >= 0),
    secondary_diagnosis_cost        REAL NOT NULL DEFAULT 0 CHECK
(secondary_diagnosis_cost >= 0),

    FOREIGN KEY (time_key) REFERENCES dim_time(time_key),
    FOREIGN KEY (ward_key) REFERENCES dim_ward(ward_key),
    FOREIGN KEY (consultant_key) REFERENCES
dim_consultant(consultant_key),
    FOREIGN KEY (diagnosis_key) REFERENCES dim_diagnosis(diagnosis_key)
);

-- ========================
-- INDEXES
-- ========================

CREATE INDEX idx_fact_admission_time
    ON fact_admission (time_key);

CREATE INDEX idx_fact_admission_ward
    ON fact_admission (ward_key);
```

```
CREATE INDEX idx_fact_admission_time_ward
    ON fact_admission (time_key, ward_key);

CREATE INDEX idx_fact_admission_consultant
    ON fact_admission (consultant_key);

CREATE INDEX idx_fact_admission_diagnosis
    ON fact_admission (diagnosis_key);

-- ========================
-- VIEWS FOR Q1 AND Q2
-- ========================

CREATE VIEW vw_q1_ward_year_quarter AS
SELECT
    w.ward_name,
    t.year_num,
    t.quarter_num,
    CASE
        WHEN SUM(f.number_of_operations) = 0 THEN NULL
        ELSE SUM(f.total_operation_charges) * 1.0 /
SUM(f.number_of_operations)
    END AS charges_per_operation,
    SUM(f.length_of_stay) AS length_of_stay,
    SUM(f.admission_cost) AS admissions_cost
FROM fact_admission f
JOIN dim_time t ON f.time_key = t.time_key
JOIN dim_ward w ON f.ward_key = w.ward_key
GROUP BY w.ward_name, t.year_num, t.quarter_num;

CREATE VIEW vw_q2_ward_year_quarter_secondarydiagnosis AS
SELECT
    w.ward_name,
    t.year_num,
    t.quarter_num,
    CASE
        WHEN SUM(f.number_of_operations) = 0 THEN NULL
        ELSE SUM(f.total_operation_charges) * 1.0 /
SUM(f.number_of_operations)
    END AS charges_per_operation,
    SUM(f.length_of_stay) AS length_of_stay,
    SUM(f.admission_cost) AS admissions_cost,
    SUM(f.secondary_diagnosis_days) AS secondary_diagnosis_days,
    SUM(f.secondary_diagnosis_cost) AS secondary_diagnosis_cost
FROM fact_admission f
JOIN dim_time t ON f.time_key = t.time_key
JOIN dim_ward w ON f.ward_key = w.ward_key
```

```
WHERE f.secondary_diagnosis_days > 0
   OR f.secondary_diagnosis_cost > 0
GROUP BY w.ward_name, t.year_num, t.quarter_num;
"""
cur.executescript(schema_sql)
conn.commit()

# Show created tables and views
print("Schema created successfully at:", db_path)
print("\nTables and views:")
for row in cur.execute("""
    SELECT name, type
    FROM sqlite_master
    WHERE type IN ('table', 'view')
    ORDER BY type, name;
"""):
    print(row)

print("\nDone.")
conn.close()
```

3. Selection and Justification of Materialized Views

3.1 Introduction

Materialized views are practical for this data warehouse because the two frequent managerial queries, Q1 and Q2, are read-intensive, repeatedly use the same fact table, join the same Time and Ward dimensions, and perform the same aggregation style by Ward, Year, and Quarter. Thus, as both Q1 and Q2 aggregate over the same grouping attributes, and Q2 is essentially a specialised version of Q1, precomputing the common aggregated results as materialised views is justified.

3.2 Calculation of Fact table(s) size

The fact table in the designed warehouse is FACT_ADMISSION, and its grain is one row per admission. Therefore, If:

- W = number of wards
- Y = number of years stored in the warehouse

Then, as each year contains 4 quarters, the **maximum number of aggregate groups** needed by Q1 and Q2 is: $W \times 4 \times Y$

So, the final cardinality is $|\boldsymbol{Q1}| \leq \boldsymbol{W} \times \boldsymbol{4} \times \boldsymbol{Y}$ and $|\boldsymbol{Q2}| \leq \boldsymbol{W} \times \boldsymbol{4} \times \boldsymbol{Y}$

Example: If the hospital has 20 wards, 5 years of data and 200,000 admissions in total then:

$$| \text{FACT\_ADMISSION} | = 200,000$$

But the maximum number of ward-year-quarter groups is only:

$$20 \times 4 \times 5 = 400$$

So, a materialized aggregate with at most 400 rows can answer questions that would otherwise require scanning and grouping 200,000 admission rows. This is a very strong justification for materializing the frequent aggregates.

3.3 Matrix Specification, including involved queries against GROUP BY and WHERE clauses: The frequent queries can be summarized in the matrix below:

Query	Dimensions involved	WHERE clause	GROUP BY	Measures / Expressions
Q1	FACT_ADMISSION, DIM_TIME, DIM_WARD	None beyond joins	Ward, Year, Quarter	SUM(total_operation_charges), SUM(number_of_operations), SUM(length_of_stay), SUM(admission_cost)
Q2	FACT_ADMISSION, DIM_TIME, DIM_WARD	admissions with secondary diagnosis	Ward, Year, Quarter	SUM(total_operation_charges), SUM(number_of_operations), SUM(length_of_stay), SUM(admission_cost), plus secondary diagnosis measures if needed

3.4 Justified choice of Materialised views

The most convenient design is to create one consolidated materialized view at the grouping level: **Ward, Year, Quarter** and store in it both:

- the overall aggregates needed for Q1.
- the secondary-diagnosis-specific aggregates needed for Q2.

This is the best because:

- Both queries aggregate by the same dimensions: **Ward, Year, Quarter**. Therefore, the same grouping can serve both queries.
- Q2 does not introduce a new grouping level. It simply focuses on admissions associated with secondary diagnosis. Therefore, it is more efficient to store both overall and secondary-diagnosis-specific aggregates in the same materialized result, rather than maintaining two largely overlapping materialized views.

- Since Q1 and Q2 are so similar, a single materialized view minimizes maintenance overhead while still accelerating both queries.

4. Implemented Materialised Views To Answer The Managers' Frequent Queries Q1 And Q2

4.1 Implementation Code for Materialised Views that answer Queries Q1 and Q2

To answer the two frequent managerial queries efficiently, the materialized view is implemented at the common aggregation level: Ward, Year, Quarter.

```
# Colab-ready sample data + materialised-view-style tables for Q1 and
Q2

import sqlite3
import pandas as pd

db_path = "/content/hospital_dw.db"
conn = sqlite3.connect(db_path)
cur = conn.cursor()

cur.execute("PRAGMA foreign_keys = ON;")

# ------------------------------------------------------------
# 1. CLEAR EXISTING SAMPLE DATA
# ------------------------------------------------------------
cur.executescript("""
DELETE FROM fact_admission;
DELETE FROM dim_diagnosis;
DELETE FROM dim_consultant;
DELETE FROM dim_ward;
DELETE FROM dim_time;

DROP TABLE IF EXISTS mv_q1_ward_year_quarter;
DROP TABLE IF EXISTS mv_q2_ward_year_quarter_secondarydiagnosis;
""")

# ------------------------------------------------------------
# 2. INSERT SAMPLE DIMENSION DATA
# ------------------------------------------------------------
cur.executemany("""
INSERT INTO dim_time (time_key, full_date, month_num, quarter_num,
year_num)
VALUES (?, ?, ?, ?, ?)
""", [
    (1, "2024-01-15", 1, 1, 2024),
    (2, "2024-02-20", 2, 1, 2024),
    (3, "2024-04-10", 4, 2, 2024),
```

```
    (4, "2025-01-12", 1, 1, 2025),
])

cur.executemany("""
INSERT INTO dim_ward (ward_key, ward_id_bk, ward_name)
VALUES (?, ?, ?)
""", [
    (1, 101, "Cardiology"),
    (2, 102, "Orthopaedics"),
])

cur.executemany("""
INSERT INTO dim_consultant (consultant_key, consultant_name)
VALUES (?, ?)
""", [
    (1, "Dr Adams"),
    (2, "Dr Brown"),
])

cur.executemany("""
INSERT INTO dim_diagnosis (diagnosis_key, diagnosis_name)
VALUES (?, ?)
""", [
    (1, "Hypertension"),
    (2, "Fracture"),
    (3, "Diabetes"),
])

# ------------------------------------------------------------
# 3. INSERT SAMPLE FACT DATA
# Grain: one row per admission
# ------------------------------------------------------------
cur.executemany("""
INSERT INTO fact_admission (
    admission_key,
    admission_id_bk,
    time_key,
    ward_key,
    consultant_key,
    diagnosis_key,
    number_of_admissions,
    length_of_stay,
    admission_cost,
    total_operation_charges,
    number_of_operations,
    secondary_diagnosis_days,
    secondary_diagnosis_cost
)
```

```
VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
""", [
    # 2024 Q1 - Cardiology
    (1, 1001, 1, 1, 1, 1, 1, 5, 1200.0, 500.0, 1, 0,   0.0),
    (2, 1002, 2, 1, 1, 3, 1, 7, 1800.0, 900.0, 2, 3, 400.0),

    # 2024 Q1 - Orthopaedics
    (3, 1003, 2, 2, 2, 2, 1, 4, 1500.0, 700.0, 1, 2, 250.0),

    # 2024 Q2 - Cardiology
    (4, 1004, 3, 1, 2, 1, 1, 6, 2100.0, 800.0, 2, 0,   0.0),

    # 2024 Q2 - Orthopaedics
    (5, 1005, 3, 2, 2, 2, 1, 9, 3200.0, 1500.0, 3, 4, 600.0),

    # 2025 Q1 - Cardiology
    (6, 1006, 4, 1, 1, 3, 1, 8, 2500.0, 1000.0, 2, 5, 900.0),
])

conn.commit()

# ------------------------------------------------------------
# 4. CREATE / REFRESH MATERIALISED-VIEW-STYLE TABLES
# ------------------------------------------------------------

# Q1: For each Ward, Year, Quarter report:
# charges per operation, length of stay, admission cost
cur.executescript("""
DROP TABLE IF EXISTS mv_q1_ward_year_quarter;

CREATE TABLE mv_q1_ward_year_quarter AS
SELECT
    w.ward_name AS ward,
    t.year_num AS year,
    t.quarter_num AS quarter,
    CASE
        WHEN SUM(f.number_of_operations) = 0 THEN NULL
        ELSE ROUND(SUM(f.total_operation_charges) * 1.0 /
SUM(f.number_of_operations), 2)
    END AS charges_per_operation,
    SUM(f.length_of_stay) AS length_of_stay,
    ROUND(SUM(f.admission_cost), 2) AS admission_cost
FROM fact_admission f
JOIN dim_time t
    ON f.time_key = t.time_key
JOIN dim_ward w
    ON f.ward_key = w.ward_key
GROUP BY
```

```
    w.ward_name,
    t.year_num,
    t.quarter_num;
""")

# Q2: For each Ward, Year, Quarter report:
# charges per operation, length of stay, admission cost
# for admissions involving secondary diagnosis
cur.executescript("""
DROP TABLE IF EXISTS mv_q2_ward_year_quarter_secondarydiagnosis;

CREATE TABLE mv_q2_ward_year_quarter_secondarydiagnosis AS
SELECT
    w.ward_name AS ward,
    t.year_num AS year,
    t.quarter_num AS quarter,
    CASE
        WHEN SUM(f.number_of_operations) = 0 THEN NULL
        ELSE ROUND(SUM(f.total_operation_charges) * 1.0 /
SUM(f.number_of_operations), 2)
    END AS charges_per_operation,
    SUM(f.length_of_stay) AS length_of_stay,
    ROUND(SUM(f.admission_cost), 2) AS admission_cost,
    SUM(f.secondary_diagnosis_days) AS secondary_diagnosis_days,
    ROUND(SUM(f.secondary_diagnosis_cost), 2) AS
secondary_diagnosis_cost
FROM fact_admission f
JOIN dim_time t
    ON f.time_key = t.time_key
JOIN dim_ward w
    ON f.ward_key = w.ward_key
WHERE f.secondary_diagnosis_days > 0
   OR f.secondary_diagnosis_cost > 0
GROUP BY
    w.ward_name,
    t.year_num,
    t.quarter_num;
""")

conn.commit()

# ------------------------------------------------------------
# 5. TEST THE MATERIALISED-VIEW TABLES
# ------------------------------------------------------------
q1_df = pd.read_sql_query("""
SELECT *
FROM mv_q1_ward_year_quarter
ORDER BY year, quarter, ward
```

```
""", conn)

q2_df = pd.read_sql_query("""
SELECT *
FROM mv_q2_ward_year_quarter_secondarydiagnosis
ORDER BY year, quarter, ward
""", conn)

print("Q1 materialised-view table:")
display(q1_df)

print("\nQ2 materialised-view table:")
display(q2_df)

# ------------------------------------------------------------
# 6. OPTIONAL: helper to refresh the materialised-view tables
# ------------------------------------------------------------
def refresh_materialized_views(connection):
    cursor = connection.cursor()
    cursor.executescript("""
    DROP TABLE IF EXISTS mv_q1_ward_year_quarter;
    CREATE TABLE mv_q1_ward_year_quarter AS
    SELECT
        w.ward_name AS ward,
        t.year_num AS year,
        t.quarter_num AS quarter,
        CASE
            WHEN SUM(f.number_of_operations) = 0 THEN NULL
            ELSE ROUND(SUM(f.total_operation_charges) * 1.0 /
SUM(f.number_of_operations), 2)
        END AS charges_per_operation,
        SUM(f.length_of_stay) AS length_of_stay,
        ROUND(SUM(f.admission_cost), 2) AS admission_cost
    FROM fact_admission f
    JOIN dim_time t
        ON f.time_key = t.time_key
    JOIN dim_ward w
        ON f.ward_key = w.ward_key
    GROUP BY
        w.ward_name,
        t.year_num,
        t.quarter_num;

    DROP TABLE IF EXISTS mv_q2_ward_year_quarter_secondarydiagnosis;
    CREATE TABLE mv_q2_ward_year_quarter_secondarydiagnosis AS
    SELECT
        w.ward_name AS ward,
        t.year_num AS year,
```

```
        t.quarter_num AS quarter,
        CASE
            WHEN SUM(f.number_of_operations) = 0 THEN NULL
            ELSE ROUND(SUM(f.total_operation_charges) * 1.0 /
SUM(f.number_of_operations), 2)
        END AS charges_per_operation,
        SUM(f.length_of_stay) AS length_of_stay,
        ROUND(SUM(f.admission_cost), 2) AS admission_cost,
        SUM(f.secondary_diagnosis_days) AS secondary_diagnosis_days,
        ROUND(SUM(f.secondary_diagnosis_cost), 2) AS
secondary_diagnosis_cost
    FROM fact_admission f
    JOIN dim_time t
        ON f.time_key = t.time_key
    JOIN dim_ward w
        ON f.ward_key = w.ward_key
    WHERE f.secondary_diagnosis_days > 0
       OR f.secondary_diagnosis_cost > 0
    GROUP BY
        w.ward_name,
        t.year_num,
        t.quarter_num;
    """)
    connection.commit()

print("\nMaterialised-view tables created and tested successfully.")
```

4.2 Results Display of the above Materialized Views Implementations

Q1 materialised-view table:

	ward	year	quarter	charges_per_operation	length_of_stay	admission_cost
0	Cardiology	2024	1	466.67	12	3000.0
1	Orthopaedics	2024	1	700.00	4	1500.0
2	Cardiology	2024	2	400.00	6	2100.0
3	Orthopaedics	2024	2	500.00	9	3200.0
4	Cardiology	2025	1	500.00	8	2500.0

Q2 materialised-view table:

	ward	year	quarter	charges_per_operation	length_of_stay	admission_cost	secondary_diagnosis_days	secondary_diagnosis_cost
0	Cardiology	2024	1	450.0	7	1800.0	3	400.0
1	Orthopaedics	2024	1	700.0	4	1500.0	2	250.0
2	Orthopaedics	2024	2	500.0	9	3200.0	4	600.0
3	Cardiology	2025	1	500.0	8	2500.0	5	900.0

Materialised-view tables created and tested successfully.